



Laboratorio No.2 Semana #13 – API Secrets y autenticación

Nombre: Alejandro Mogollón Rodríguez **Carné:** 1132125

Objetivo: Practicar el consumo de una API REST autenticada y aplicar una buena práctica básica de seguridad: evitar colocar credenciales directamente en el código fuente. El estudiante utilizará Postman, una aplicación C# y GitHub.

Parte 1 – Probar autenticación con Postman

Para esta práctica se utilizará DummyJSON, una API pública de pruebas que permite realizar un inicio de sesión y obtener un token de acceso.

Documentación: <https://dummyjson.com/docs/auth> <https://dummyjson.com/docs/users>

1. Crear la solicitud de inicio de sesión

- Abra Postman y cree una nueva colección llamada: MIA – Semana 13.
- Cree una solicitud llamada: Login.
- Seleccione el método POST.
- Utilice el endpoint: `https://dummyjson.com/auth/login`
- En Body seleccione raw y JSON.
Ingrese las credenciales de prueba: Username: emilys | Password: emilyspass.
`{ "username": "emilys", "password": "emilyspass" }`
- Ejecute la solicitud y localice en la respuesta el campo accessToken.

2. Utilizar token

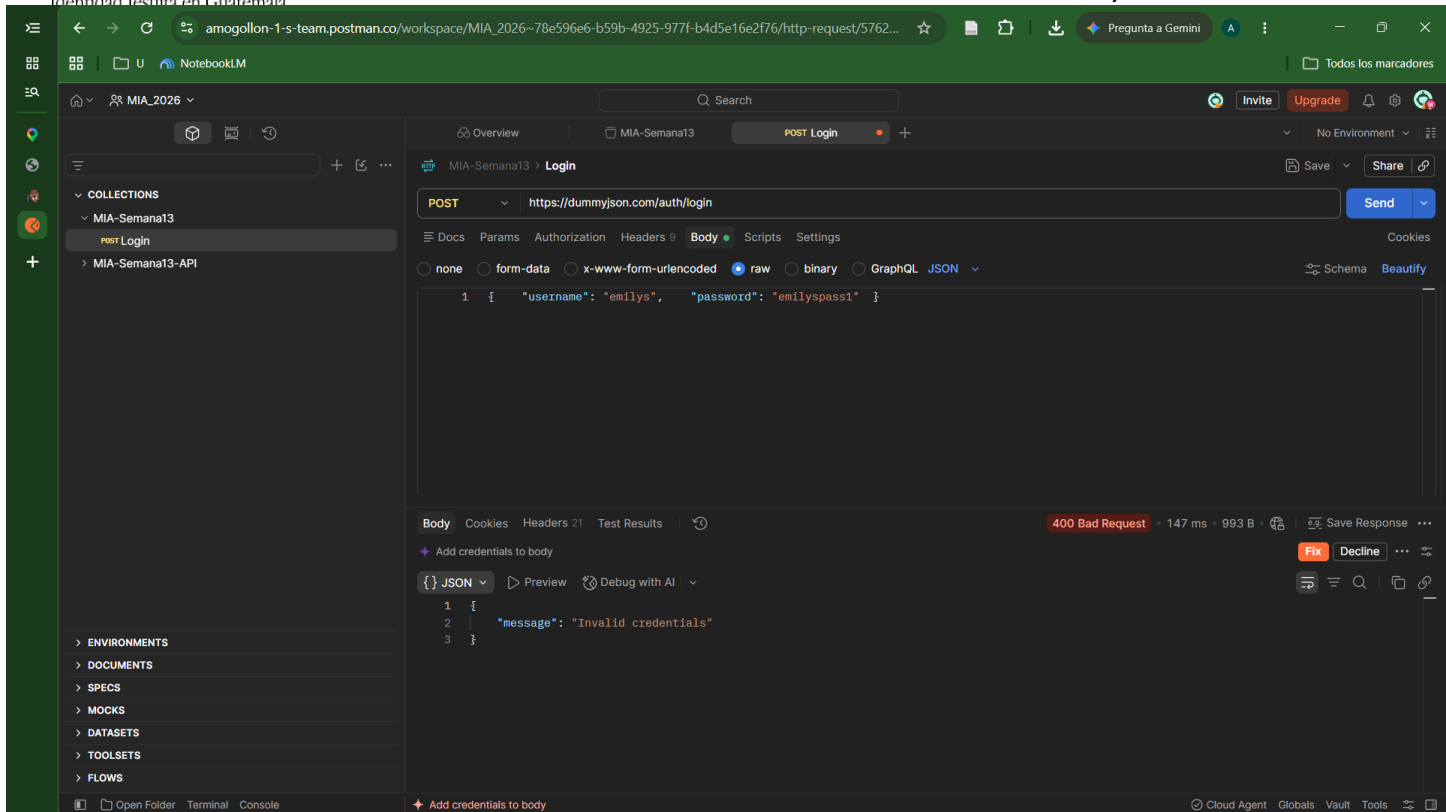
- Cree una segunda solicitud llamada: Usuario autenticado.
- Seleccione GET.
- Utilice: `https://dummyjson.com/user/me`
- En Authorization seleccione Bearer Token.
- Copie el accessToken obtenido en la solicitud Login.
- Ejecute la solicitud.
- Verifique que la API devuelve la información del usuario autenticado.

3. Comprobar qué ocurre sin autenticación

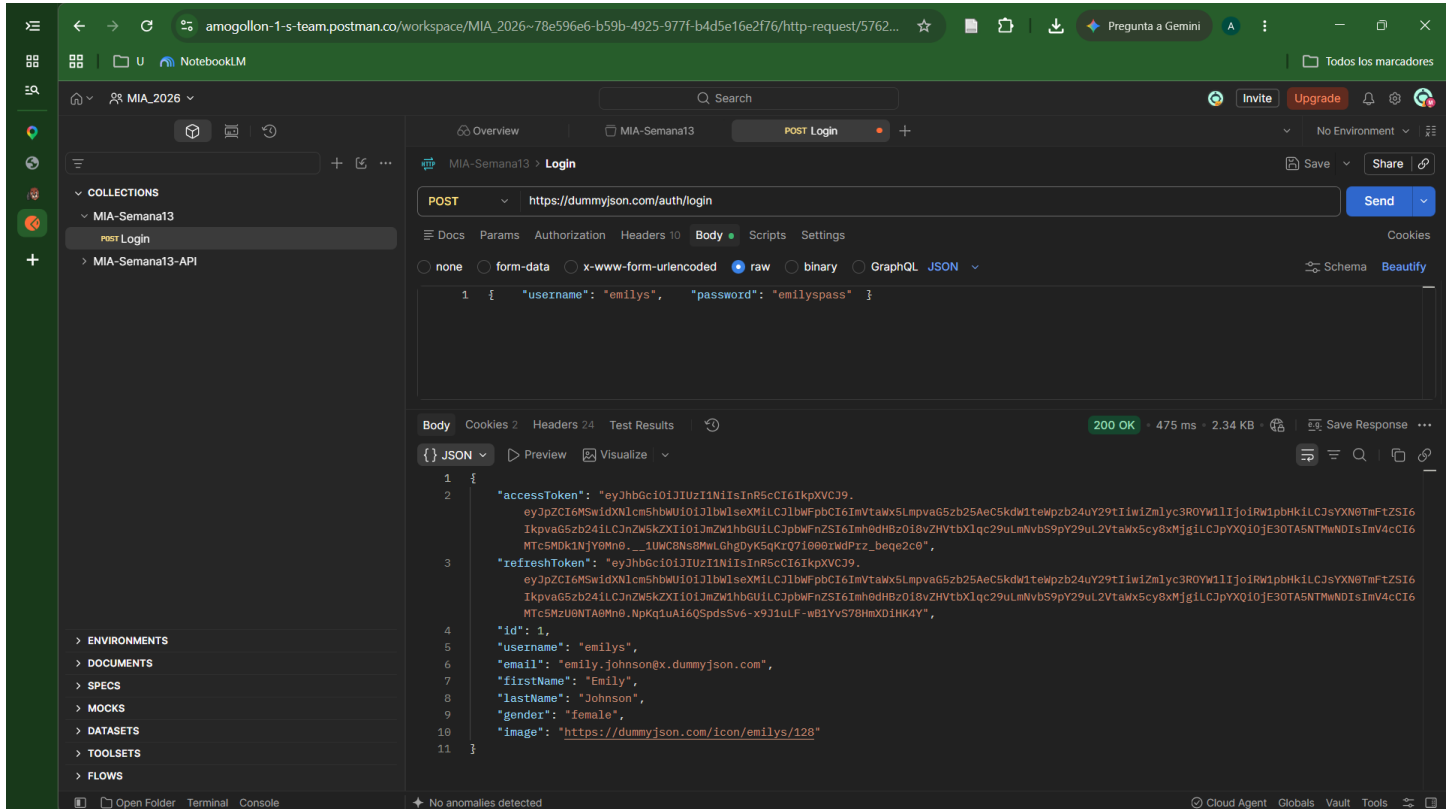
- Duplique la solicitud Usuario autenticado.
- Quite el token.
- Ejecute nuevamente la solicitud.
- Compare la respuesta con la solicitud autenticada.
- Registre el código HTTP y explique la diferencia.

4. Evidencia

- Captura de Login mostrando la respuesta.
Prueba con una contraseña errónea



Acceso correcto





- Captura de la solicitud autenticada.

The screenshot shows the Postman interface with a collection named 'MIA-Semana13' and a sub-collection 'MIA-Semana13-API'. The selected request is 'GET Usuario Autenticado'. The URL is 'https://dummyjson.com/user/me'. The 'Authorization' tab is active, showing 'Bearer Token' as the auth type. The 'Token' field contains a masked Bearer token. The 'Body' tab shows a JSON response with status '200 OK', indicating a successful authentication request.

- Captura de la prueba sin token.

The screenshot shows the Postman interface with the same collection and sub-collection. A new request, 'GET Usuario Autenticado Copy', is selected. The URL is 'https://dummyjson.com/user/me'. The 'Authorization' tab is active, but the 'Token' field is empty. The 'Body' tab shows a JSON response with status '401 Unauthorized', indicating that the request failed due to missing authentication credentials.



Con token erróneo

The screenshot shows the Postman interface with a collection named 'MIA-Semana13' and a sub-collection 'Usuario Autenticado'. The selected request is 'GET Usuario Autenticado Copy' with the URL 'https://dummyjson.com/user/me'. The 'Authorization' tab is active, showing 'Bearer Token' as the auth type. The token field contains a masked value. The response is a 401 Unauthorized status with a message: 'Invalid/Expired Token!'. The bottom status bar indicates 'No anomalies detected'.

- Indique qué función cumple el Bearer Token.
Se encarga de autorizar el acceso sin necesidad de ingresar user y password de nuevo, su nombre indica “permitir ingreso al portador de este token”.

La documentación de DummyJSON muestra el flujo de login, la obtención del accessToken y su envío mediante el encabezado Authorization: Bearer.



Parte 2 – Proteger la Connection String de Azure Storage

Utilice como base el programa C# desarrollado anteriormente para conectarse a Azure Storage. No se requiere crear nuevamente el Storage Account

5. Localizar la Connection String

- Abra el proyecto C# de Azure Storage.
- En el Explorador de soluciones, abra el archivo donde actualmente está escrita la Connection String. En un proyecto sencillo normalmente será Program.cs.
- Busque la variable que contiene la Connection String.
- Identifique el valor que comienza con información como DefaultEndpointsProtocol=...

6. Reemplazar la Connection String

Reemplace el valor escrito directamente en el programa por una lectura de una variable de entorno llamada:

AZURE_STORAGE_CONNECTION_STRING

La idea es que el programa siga utilizando la misma variable para conectarse a Azure, pero ahora el valor será obtenido desde el entorno y no desde el código

7. Crear la variable para ejecutar localmente

- En la carpeta principal del proyecto, cree un archivo llamado .env.
- Dentro del archivo agregue la variable:
`AZURE_STORAGE_CONNECTION_STRING=TU_CONNECTION_STRING`
- Reemplace SU_CONNECTION_STRING por la Connection String de su Storage Account.
- Verifique que el programa lea la variable desde el .env y ejecute nuevamente el programa.
- Compruebe que el programa continúa funcionando correctamente.
- No publique el archivo .env en GitHub**, ya que contiene la credencial real.

8. Crear el Secret en Github

- Ingresa al repositorio de su proyecto en GitHub.
- Seleccione Settings.
- En el menú lateral seleccione Secrets and variables.
- Seleccione Actions.
- Seleccione New repository secret.
- En Name escriba: `AZURE_STORAGE_CONNECTION_STRING`
- En Secret coloque la Connection String de Azure Storage.x
- Seleccione Add secret.



Archivo que ya estaba en el container

Inicio > miastorage2026

miastorage2026 | Explorador de almacenamiento

Cuenta de almacenamiento

Buscar

Información general
Registro de actividad
Etiquetas
Diagnosticar y solucionar problemas
Control de acceso (IAM)
Migración de datos
Eventos
Explorador de almacenamiento
Storage Mover
Soluciones de asociados
Visualizador de recursos
Almacenamiento de datos
Seguridad y redes
Redes
Front Door y CDN
Claves de acceso
Firma de acceso compartido

miastorage2026

- Favoritos
- Vistos recientemente
- Contenedores de blobs
 - slogs
 - documentos
 - miarchivos
- Ver todo
- Recursos compartidos de archivos clásicos
 - Colas
 - Tablas

+ Agregar directorio | Cargar | Cambiar nivel de acceso | Actualizar | Eliminar | Copiar | Pegar

Contenedores de blobs > miarchivos

Método de autenticación: Clave de acceso (Cambiar a la cuenta de usuario de Microsoft Entra)

Agregar filtro

Buscar blobs por prefijo (distingue mayúsculas de minúsculas)

Solo mostrar blobs activos

Mostrando todos los elementos 1

	Nombre	Última modificación	Nivel de acceso	Tipo de blob	Tamaño	Estado de conc.
	pruebas.bl...	2/10/2026, 18:37:41	Frecuente (inferido)	Blob en bloques	93 B	Disponible

Agree o quite favoritos presionando Ctrl+Shift+F

Eliminación de archivo en container

File Edit Selection View Go Run Terminal Help

url-mia-2026

Explor

- url-mia-2026
 - Semana_03
 - Semana_04
 - Semana_05
 - Semana_06
 - Semana_07
 - Semana_08
 - Semana_09
 - Semana_10
 - Semana_11
 - Semana_12
 - lab12
 - bin
 - obj
 - .env
 - .env.production
 - lab12.csproj
 - MIA_AzureBlob.cs
 - MIA_Laboratorio_No_2_Sema...
 - Semana_13
 - Semana_14
 - Semana_15
 - Semana_16
 - Semana_17
- Outline
- Timeline
- C# Project Details

pruebas_blob.txt

Readme.md
.gitkeep
12.txt
hola-azure.txt
Lab12 - Cloud Storage (Azure)...

Semana_12 > lab12 > MIA_AzureBlob.cs

```
1 using Azure.Storage.Blobs;
2 using DotNetEnv;
3
4 Env.Load();
5 // Obtener Connection String desde la variable de entorno
6 string? connectionString = Environment.GetEnvironmentVariable("AZURE_STORAGE_CONNECTION_STRING");
7
8 if (string.IsNullOrEmpty(connectionString))
9 {
10     Console.WriteLine("No se encontró la Connection String.");
11     return;
12 }
13
14 // Nombre real de tu container
15 string nombrecontainer = "miarchivos";
16
17 try
18 {
19     // Conexión con Azure Blob Storage
20     BlobServiceClient servicio =
21         new BlobServiceClient(connectionString);
```

Problems Output Debug Console Terminal Ports Spell Checker 106

1. Subir archivo
2. Listar archivos
3. Descargar archivo
4. Eliminar archivo
5. Salir
Seleccione una opción: 4

ELIMINAR ARCHIVO

Ingrese el nombre del archivo: pruebas_blob.txt
¿Está seguro de eliminar 'pruebas_blob.txt'? (S/N): s

Archivo eliminado correctamente.

Presione una tecla para continuar...

Ln 15, Col 40 Spaces: 4 UTF-8 with BOM CRLF C# Go Live



Microsoft Azure portal showing the 'miastorage2026' storage account. The interface displays the 'Explorador de almacenamiento' (Storage Explorer) with a sidebar on the left containing navigation options like 'Información general', 'Registro de actividad', 'Etiquetas', 'Diagnosticar y solucionar problemas', 'Control de acceso (IAM)', 'Migración de datos', 'Eventos', 'Explorador de almacenamiento', 'Storage Mover', 'Soluciones de asociados', 'Visualizador de recursos', 'Almacenamiento de datos', 'Seguridad y redes', 'Redes', 'Front Door y CDN', 'Claves de acceso', and 'Firma de acceso compartido'.

The main area shows the 'miastorage2026' storage account with a search bar and a list of containers. The 'Contenedores de blobs' section is active, showing a list of blobs. The 'Método de autenticación' is set to 'Clave de acceso'. The 'Agregar filtro' button is visible. The 'Buscar blobs por prefijo' field is empty. The 'Solo mostrar blobs activos' dropdown is set to 'Solo mostrar blobs activos'.

The table below shows the results of the search:

Nombre	Última modificación	Nivel de acceso	Tipo de blob	Tamaño	Estado de conc.
No se encontraron elementos					

Resubiendo archivo

Visual Studio Code editor showing the 'MIA_AzureBlob.cs' file. The code is in C# and uses the Azure Storage SDK. The file is located in the 'url-mia-2026' project, under the 'Semana_12' folder, in the 'lab12' subfolder. The code is as follows:

```
1 using Azure.Storage.Blobs;
2 using DotNetEnv;
3
4 Env.Load();
5 // Obtener Connection String desde la variable de entorno
6 string? connectionString = Environment.GetEnvironmentVariable("AZURE_STORAGE_CONNECTION_STRING");
7
8 if (string.IsNullOrEmpty(connectionString))
9 {
10     Console.WriteLine("No se encontró la Connection String.");
11     return;
12 }
13
14 // Nombre real de tu container
15 string nombreContainer = "miaarchivos";
16
17 try
18 {
19     // Conexión con Azure Blob Storage
20     BlobServiceClient servicio =
21         new BlobServiceClient(connectionString);
22 }
```

The 'Terminal' window shows the output of the application:

```
1. Subir archivo
2. Listar archivos
3. Descargar archivo
4. Eliminar archivo
5. Salir
Seleccione una opción: 1

SUBIR ARCHIVO

Ingrese la ruta del archivo: C:\url-mia-2026\Semana_12\lab12\pruebas_blob.txt

Archivo subido correctamente.
Nombre: pruebas_blob.txt

Presione una tecla para continuar...
```



Archivo resubido

Inicio > miastorage2026

miastorage2026 | Explorador de almacenamiento

Cuenta de almacenamiento

Buscar

Información general
Registro de actividad
Etiquetas
Diagnosticar y solucionar problemas
Control de acceso (IAM)
Migración de datos
Eventos
Explorador de almacenamiento
Storage Mover
Soluciones de asociados
Visualizador de recursos
Almacenamiento de datos
Seguridad y redes
Redes
Front Door y CDN
Claves de acceso
Firma de acceso compartido

miastorage2026

- Favoritos
- Vistos recientemente
- Contenedores de blobs
 - \$logs
 - documentos
 - miarchivos
- Ver todo
- Recursos compartidos de archivos clásicos
 - Colas
 - Tablas

+ Agregar directorio | Cargar | Cambiar nivel de acceso | Actualizar | Eliminar | Copiar | Pegar

Contenedores de blobs > miarchivos

Método de autenticación: Clave de acceso (Cambiar a la cuenta de usuario de Microsoft Entra)

Agregar filtro

Buscar blobs por prefijo (distingue mayúsculas de minúsculas)

Solo mostrar blobs activos

Mostrando todos los elementos 1

	Nombre	Última modificación	Nivel de acceso	Tipo de blob	Tamaño	Estado de conc.
	pruebas_bl...	2/10/2026, 18:40:03	Frecuente (inferido)	Blob en bloques	93 B	Disponible

Agree o quite favoritos presionando Ctrl+Shift+F

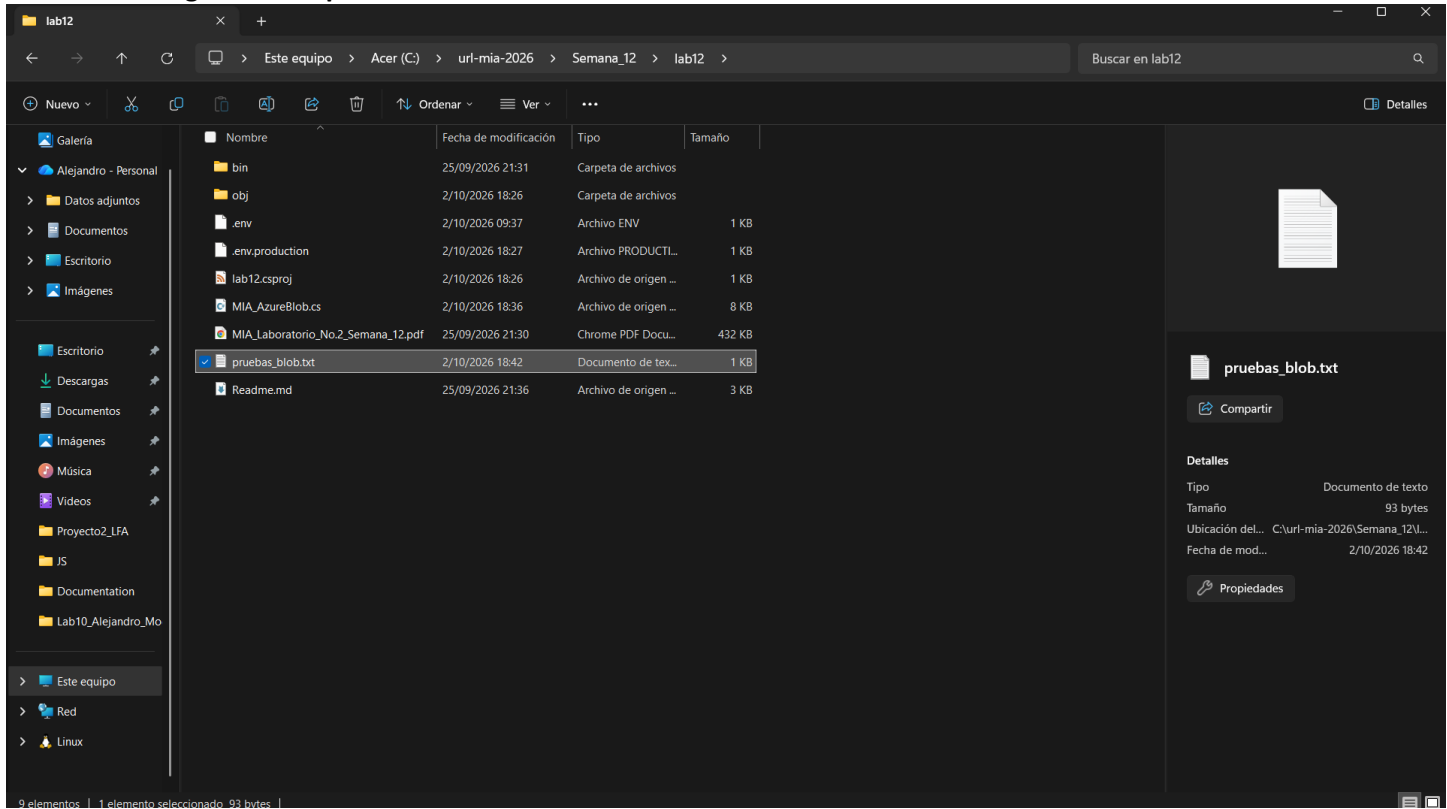
Descarga de archivo desde azure

```
File Edit Selection View Go Run Terminal Help
url-mia-2026
Semana_03
Semana_04
Semana_05
Semana_06
Semana_07
Semana_08
Semana_09
Semana_10
Semana_11
Semana_12
  bin
  obj
  .env
  .env.production
  lab12.csproj
  MIA_AzureBlob.cs
  MIA_Laboratorio_No2_Sema...
  pruebas_blob.txt
  README.md
  .gitkeep
  12.txt
  hola-azure.txt
  Lab12 - Cloud Storage (Azure)....
Semana_13
Semana_14
Semana_15
Semana_16
Semana_17
Outline
Timeline
C# Project Details

MIA_AzureBlob.cs
1 using Azure.Storage.Blobs;
2 using DotNetEnv;
3
4 Env.Load();
5 // Obtener Connection String desde la variable de entorno
6 string? connectionString = Environment.GetEnvironmentVariable("AZURE_STORAGE_CONNECTION_STRING");
7
8 if (string.IsNullOrEmpty(connectionString))
9 {
10     Console.WriteLine("No se encontró la Connection String.");
11     return;
12 }
13
14 // Nombre real de tu container
15 string nombreContainer = "miarchivos";
16
17 try
18 {
19     // Conexión con Azure Blob Storage
20     BlobServiceClient servicio =
21         new BlobServiceClient(connectionString);
22 }
23 catch { }
24
25 2. Listar archivos
26 3. Descargar archivo
27 4. Eliminar archivo
28 5. Salir
29 Seleccione una opción: 3
30
31 DESCARGAR ARCHIVO
32
33 Ingrese el nombre del archivo: pruebas_blob.txt
34 Ingrese la carpeta de destino: C:\url-mia-2026\Semana_12\lab12
35
36 Archivo descargado correctamente.
37 Ubicación: C:\url-mia-2026\Semana_12\lab12\pruebas_blob.txt
38
39 Presione una tecla para continuar...
```



Archivo descargado en carpeta



Entrega

Dentro del repositorio de GitHub, cree la carpeta semana-13/lab2/ y coloque en ella un PDF con la solución del laboratorio.

Incluya también los archivos de respaldo generados durante la práctica, así como los pantallazos que evidencien las pruebas realizadas.

Enviar link de github a plataforma de la universidad.